

AI Concepts in LLM-Based Threat Modeling

A detailed guide to Large Language Models, NLP, Embeddings, Retrieval-Augmented Generation (RAG), and Graph Neural Networks

Based on: Facilitating Threat Modeling by Leveraging Large Language Models

Elsharef, Zeng & Gu — AISCC 2024

1. Large Language Models (LLMs)

A Large Language Model is a neural network trained on vast quantities of text to understand and generate human language. The model at the heart of this paper is Meta's Llama 2, used in its 7-billion and 13-billion parameter variants, fine-tuned for dialogue (Llama 2-CHAT) and run on a T4 GPU in Google Colab. The sections below walk through every stage of how it works, from raw text input to generated answer.

1.1 End-to-End Pipeline

The diagram below shows the complete outer loop of an LLM. A string of text goes in; one new token comes out; that token is appended and the whole process repeats until the answer is complete.

Figure 1. End-to-end flow of a large language model

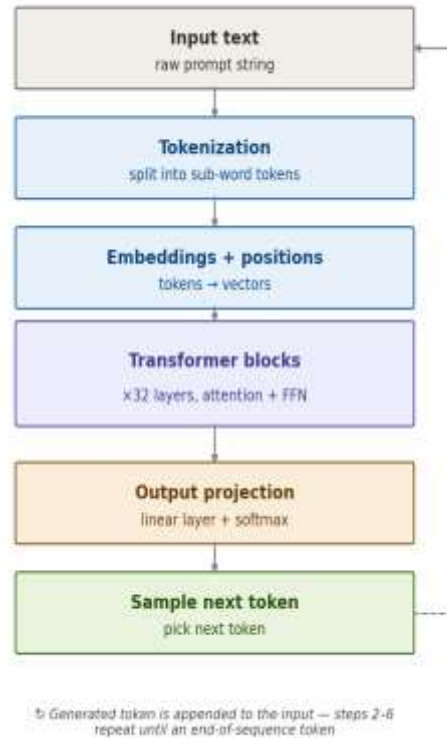


Figure 1. End-to-end LLM pipeline — the autoregressive generation loop

Step 1 — Input text

The raw prompt fed to Llama 2 in this paper is a combination of: (a) retrieved document chunks from the Pinecone vector database and (b) the security engineer's natural-language question. This combined text can be hundreds to thousands of characters long.

Step 2 — Tokenization

The text is split into **sub-word tokens** by a Byte-Pair Encoding (BPE) tokenizer. Rather than treating every word as a single unit, BPE builds a vocabulary of common character sequences. A rare word like 'Modbus' might become one token; 'unrecognized' might split into 'un' + 'recogn' + 'ized'. Llama 2 uses a vocabulary of ~32,000 tokens. Each token is assigned an integer ID.

Step 3 — Embeddings + Positional Encoding

Each integer token ID is looked up in a learned embedding table, producing a dense vector of floating-point numbers (4,096 dimensions for the 7B model; larger for 13B). These embedding vectors encode meaning — tokens with related meanings cluster together in the high-dimensional space. Because attention has no inherent notion of word order, positional information is added to each embedding so the model can distinguish 'the firmware updates the key' from 'the key updates the firmware'. Llama 2 uses Rotary Position Embeddings (RoPE) for this.

Step 4 — Transformer Blocks (×32)

The sequence of embedding vectors passes through 32 identical transformer blocks (in the 7B model). Each block refines every token's vector representation using context from all surrounding tokens. This is where essentially all the 'thinking' happens. The next section zooms into a single block.

Step 5 — Output Projection

After the final transformer block, each token position carries a richly context-aware vector. The last position's vector is projected through one final linear layer into a vector of size 32,000 (one number per vocabulary token). A softmax function converts these raw scores into a probability distribution: 'given everything so far, how likely is each possible next word?'

Step 6 — Sample Next Token

A token is sampled from this distribution. Greedy decoding always picks the highest-probability token; temperature or top-p sampling introduces controlled randomness, which is why the same prompt can produce slightly different outputs across runs. The selected token is appended to the sequence, and the pipeline runs again from Step 2 — this continues until an end-of-sequence token is produced or a maximum length is reached. This is why response times in the paper ranged from 25 to 50 seconds: each of ~74 output tokens requires a full forward pass through all 32 transformer blocks of a billions-parameter model.

2. Inside a Transformer Block

Every one of the 32 blocks has exactly the same internal structure, shown below. Only the learned weight matrices differ between blocks.

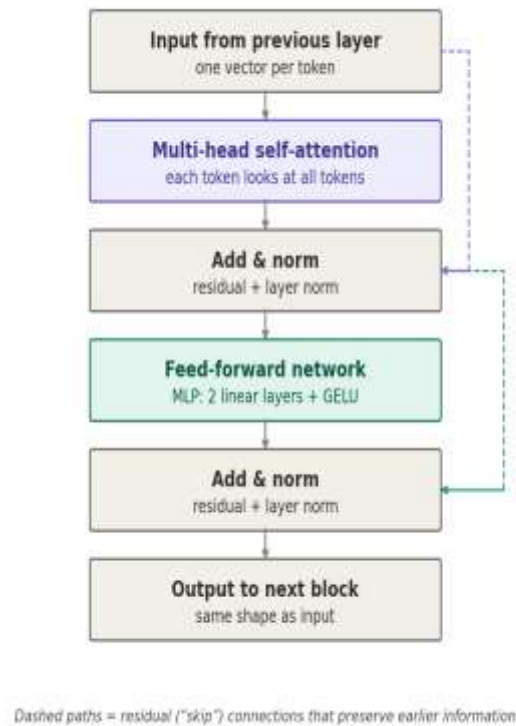
Figure 2. Inside a single transformer block (repeats x32)

Figure 2. Internal structure of a single transformer block — repeats x32

Multi-Head Self-Attention

This is the defining mechanism of the transformer architecture. For every token, three linear projections produce a Query vector (Q), a Key vector (K), and a Value vector (V). The model computes the dot product of each token's Q against every other token's K, scales and softmaxes these scores into attention weights, then produces a weighted sum of the V vectors. The result for each token is a new vector that blends information from all other tokens in proportion to how relevant they were deemed to be.

'Multi-head' means this entire process runs several times in parallel — for example, 32 heads in the 7B model — each with different learned projections. One head might learn to track syntactic subject-verb relationships; another might learn to connect a technical term to its definition earlier in the document. The outputs of all heads are concatenated and projected back to the model's hidden dimension.

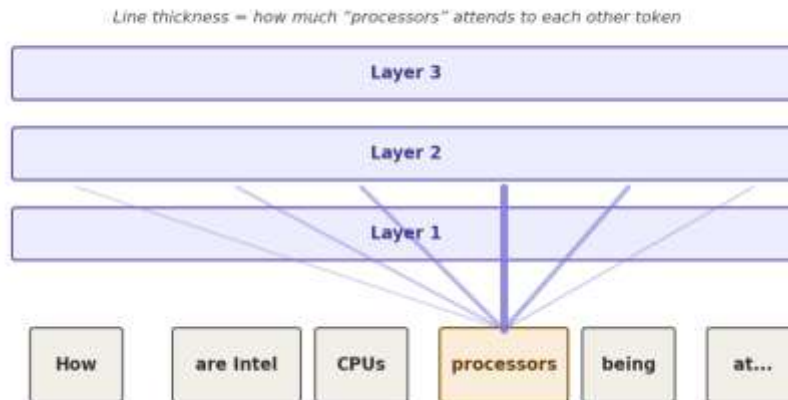
Figure 3. Self-attention: one token attending to all others

Figure 3. Self-attention visualised: line thickness shows how much 'processors' attends to each other token

Add & Norm (Residual Connection)

After the attention sub-layer, the output is added back to the original input of that sub-layer (the dashed skip connection in Figure 2). This residual connection means that if a layer learns nothing useful, it can at least pass the original signal through unchanged — vital for training 32+ layer networks without losing gradients. Layer normalisation is then applied to stabilise the scale of activations.

Feed-Forward Network (FFN)

Each token's vector — independently, no longer mixing across tokens — passes through a small multi-layer perceptron: a linear layer that expands the dimension (to ~11,000 for 7B), a GELU non-linear activation, and a second linear layer that projects back down. If attention is the 'gather context' step, the FFN is the 'process and store facts' step; research suggests much of the factual knowledge encoded in LLMs lives in the FFN weights.

Add & Norm (second)

The FFN's output is added to its input (the second dashed skip connection) and normalised, producing the block's final output — a tensor of the same shape as the input, ready to be passed to the next block.

3. NLP Techniques — KeyBERT & NLTK

Before the LLM is involved at all, Task 2 of the paper uses NLP techniques to extract the most representative keywords from design-document PDFs. These keywords are then used to query the National Vulnerability Database (NVD) and retrieve relevant CVEs. The pipeline is shown below.

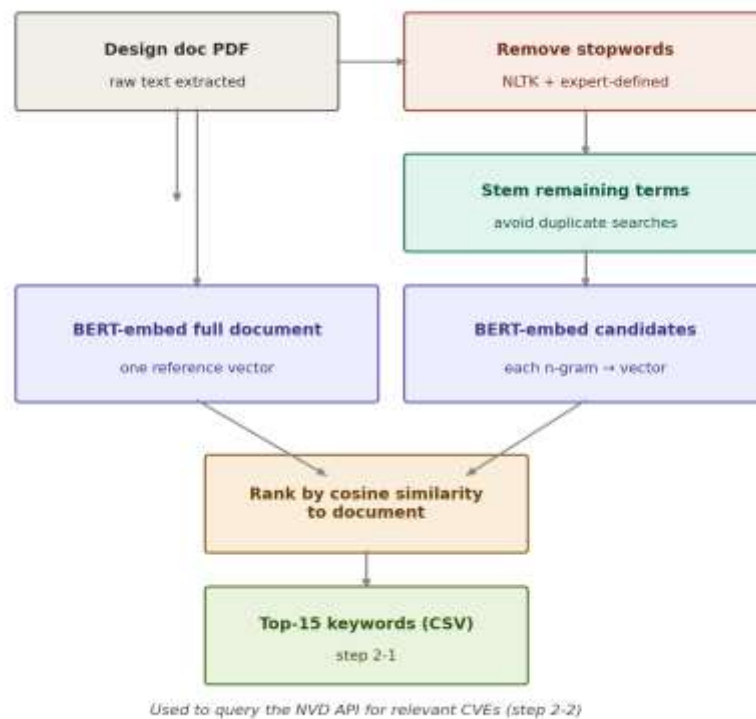
Figure 4. KeyBERT keyword-extraction pipeline (Task 2)

Figure 4. KeyBERT keyword-extraction pipeline used in Task 2 of the paper

Stopword Removal

Two layers of stopwords are stripped out before any keyword scoring:

- NLTK English stopwords — the standard list of ~180 high-frequency but low-information words such as 'the', 'and', 'it', 'are', 'that'.
- Expert-defined stopwords — a custom list of words that appear frequently in security writing but are too generic to identify a specific design: 'attack', 'hacker', 'security', 'cyber', 'architecture', 'secure', etc. Every security document would surface these, so they carry no discriminating signal.

Stemming via NLTK

NLTK's Porter stemmer reduces words to a common root form — 'authenticating', 'authentication', and 'authenticate' all become 'authent-'. This prevents the NVD query from being run multiple times with near-identical search terms, avoiding redundant results in the vulnerability dataset.

BERT Embeddings and Cosine Similarity (KeyBERT)

KeyBERT uses a BERT (Bidirectional Encoder Representations from Transformers) model to embed both the full document and every candidate keyword or key-phrase into the same high-dimensional vector space. BERT is itself a transformer-based model, but trained to produce fixed-size meaning vectors rather than to generate text.

Cosine similarity measures the angle between two vectors in this space — a score of 1 means the vectors point in the same direction (very similar meaning); 0 means they are orthogonal (unrelated). KeyBERT ranks every candidate phrase by how similar its embedding is to the whole-document embedding, and the top 15 (top_n=15) are kept. Intuitively: which short phrases best capture what the whole document is about?

The paper also allows keyphrase_ngram_range to extract multi-word phrases (e.g., 'key exchange protocol', 'trust domain extensions'), giving more specific and contextually meaningful search terms than single words alone.

4. Embeddings and Vector Databases

What are Embeddings?

An embedding is a vector of floating-point numbers that represents the meaning of a piece of text in a high-dimensional space. Unlike keyword matching, embeddings capture semantic meaning — the phrase 'private key compromise' and the phrase 'secret key exposure' will have very similar embedding vectors even though they share no words.

In this paper, embeddings are computed twice: once by KeyBERT's BERT model (for keyword extraction in Task 2), and once by the RAG system's embedding model (for indexing document chunks and retrieving relevant ones at query time in both tasks).

Pinecone Vector Database

A vector database like Pinecone is fundamentally different from a traditional SQL database. Instead of exact-match queries ('WHERE id = 5'), Pinecone answers nearest-neighbour queries: 'give me the stored vectors whose direction is most similar to this query vector.' This is implemented efficiently with approximate nearest-neighbour algorithms (e.g., HNSW graphs) that can search millions of vectors in milliseconds.

In the paper, every 500-character chunk of every design-document PDF is embedded and stored in a Pinecone index. For Task 2, each CVE description from the NVD is similarly embedded and stored. When a question is asked, its embedding is compared against the index and the most relevant chunks are retrieved.

5. Retrieval-Augmented Generation (RAG)

RAG is the central architectural innovation of the paper's threat modeling system. It solves a fundamental problem: Llama 2's weights were frozen at training time and contain no knowledge of this company's specific design documents or this week's new CVEs. RAG injects that knowledge at query time, without retraining.

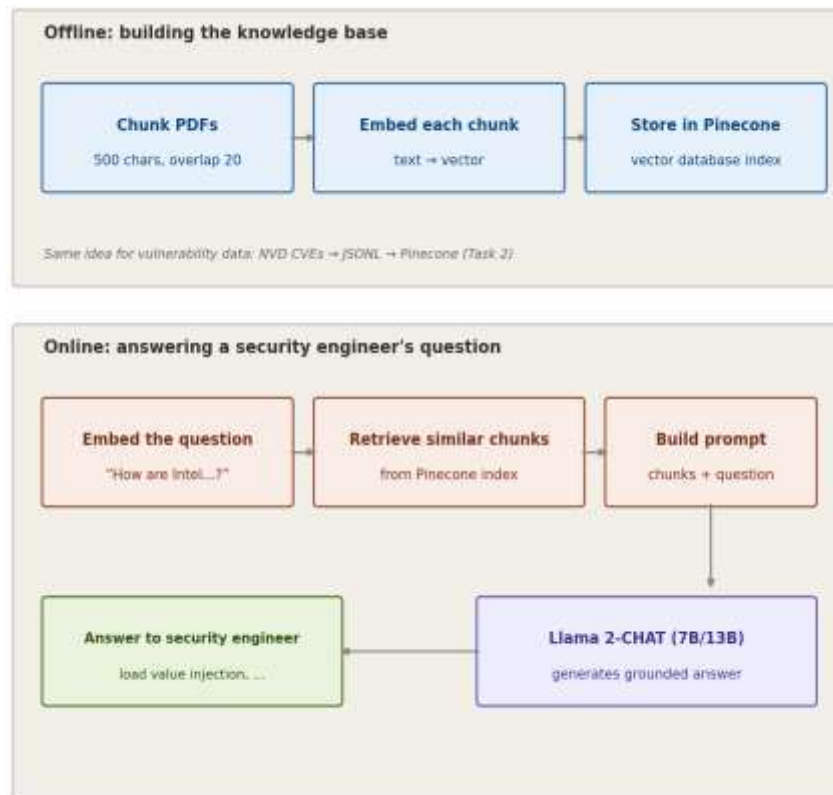
Figure 5. Full retrieval-augmented generation pipeline

Figure 5. Full retrieval-augmented generation pipeline — offline indexing and online querying

Offline Phase — Building the Knowledge Base

This phase runs once whenever the source documents change:

- **Chunk PDFs** — LangChain's RecursiveCharacterTextSplitter divides each PDF into 500-character chunks with 20 characters of overlap, so sentences straddling a boundary are not lost in both halves. Metadata (document name, page number) is attached to each chunk.
- **Embed each chunk** — an embedding model converts each text chunk into a dense vector.
- **Store in Pinecone** — the vectors and their metadata are upserted into a Pinecone index, ready for millisecond-speed similarity searches.
- **For Task 2**, vulnerability data follows an equivalent path: keywords from design docs → NVD API query → CVE descriptions formatted as JSONL → embedded → Pinecone.

Online Phase — Answering a Question

When a security engineer submits a question (e.g., 'How are Intel processors being targeted?'):

- **Embed the question** — the same embedding model converts the question into a vector.

- Retrieve similar chunks — Pinecone returns the top-k stored chunks whose vectors are closest to the question vector. These are the passages from the design documents (or CVE descriptions) most semantically relevant to the question.
- Build augmented prompt — the retrieved chunks are prepended to the question to form a prompt: 'Here is relevant context: [chunks]. Now answer: [question].'
- Llama 2 generates the answer — conditioned on both the retrieved context and the question, Llama 2 generates a grounded, specific answer rather than a generic one from its training data.

Why RAG Outperformed the Base LLM

Table IV of the paper shows a striking contrast. The base Llama 2 produced a 253-word response to 'How are Intel processors being targeted?' that included general discussion of Spectre/Meltdown, firmware vulnerabilities, and supply-chain attacks — all from training data, none specific to the TDX design document under review. The RAG-enhanced version produced a 74-word answer citing 'load value injection, improper isolation of shared resources, and sequence of processor instructions leading to unexpected behaviour' — specific CVEs retrieved from the NVD that were relevant to TDX.

RAG also produced the paper's 'awareness of honesty' behaviour: when retrieved chunks contained no relevant information, the model correctly responded 'I don't know the answer to your question' rather than hallucinating a plausible-sounding but incorrect response.

6. Graph Neural Networks (GNNs) — Concept and Relevance

Graph Neural Networks (GNNs) are not used in this paper. This section explains what they are, why the paper's architecture does not require them, and where they could plausibly fit in a future extension.

Figure 6. How a Graph Neural Network passes messages between nodes

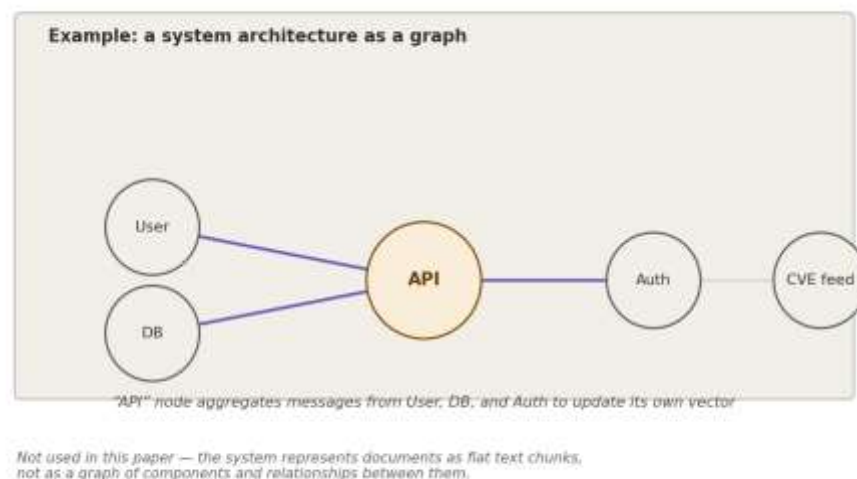


Figure 6. GNN message-passing concept — each node aggregates information from its neighbours to update its own representation

What is a GNN?

A Graph Neural Network operates on data that is naturally structured as nodes and edges — not flat sequences like text. Each node starts with a feature vector (e.g., 'this is a database node that stores customer records; it has two open ports'). In each round of message passing, every node gathers the feature vectors of its directly connected neighbours, combines them with its own current state using learned weights, and produces an updated feature vector. After several rounds, each node's vector encodes not just its own properties but information about its local neighbourhood — two, three, or more hops away.

GNNs are the standard tool for tasks where structure matters: predicting molecule properties (atoms as nodes, bonds as edges), detecting bots in social networks (users as nodes, follows as edges), or predicting which component in a network topology is most vulnerable (services as nodes, communication channels as edges).

Why This Paper Does Not Use a GNN

Both Task 1 and Task 2 treat the input as unstructured text — PDF design documents and CVE prose descriptions. The knowledge base is a flat collection of 500-character text chunks in a vector database, retrieved purely by semantic similarity. There is no explicit graph of 'this API service talks to this database, which is reachable from this external network zone.' Structural relationships must be inferred implicitly by Llama 2 from prose descriptions in the document.

Where a GNN Could Fit in a Future Extension

STRIDE-based threat modelling — which the paper targets — fundamentally operates on Data Flow Diagrams (DFDs): literal graphs of processes, data stores, external entities, and trust boundaries connected by data-flow edges. If a future system could parse an architecture diagram into a machine-readable graph (rather than just prose describing the architecture), a GNN could learn to flag structurally high-risk nodes. For example, a process node that sits at the intersection of an external-facing trust boundary and a data store containing secrets is structurally suspicious regardless of the prose description. A GNN trained on labelled DFDs could score each node for attack-surface risk, complementing the LLM's natural-language reasoning. This would be a new component layered on top of the RAG pipeline — not a replacement for it.

7. Summary — The Complete AI Stack

The table below maps every AI/ML concept in this paper to the role it plays in the threat modelling system:

NLTK + stemming

Role: NLP preprocessing

Remove stopwords and reduce words to root forms before keyword extraction in Task 2

KeyBERT + BERT embeddings

Role: NLP — keyword extraction

Embed candidate phrases and whole document; rank by cosine similarity to find top-15 representative keywords

Embeddings (vectors)

Role: Representation

Convert text chunks and questions into high-dimensional vectors that capture semantic meaning

Pinecone vector database

Role: Knowledge base

Store all chunk embeddings; answer nearest-neighbour queries at inference time

RAG framework

Role: Architecture

Retrieve relevant context at query time; inject it into the prompt so Llama 2 reasons over specific, current information

Llama 2-CHAT 7B/13B

Role: LLM — generation

Transformer-based model that generates grounded natural-language answers to threat modelling questions

GNN

Role: Not used in this paper

Potential future component: could score nodes in a parsed Data Flow Diagram by structural attack-surface risk

End of Document